Prime Land AI Assistant

Engineering Report

1. Executive Summary

Total Queries	20
Correction Frequency	1.00
Avg Confidence Gain	0.00
Avg RAG Latency	7.92
AVG CRAG Latency	7.05

Table 1.1 CRAG and RAG Evaluation Summary

This report evaluates the performance of a standard Retrieval-Augmented Generation (RAG) pipeline against a Corrective RAG(CRAG) architecture across 20 domain-specific queries. The evaluation focused on correction frequency, confidence improvement, and latency under a fixed confidence threshold of 0.6.

Results indicate that all RAG responses produced confidence scores significantly below the benchmark threshold (average 0.2), leading to a correction frequency of 100% in the CRAG pipeline. However, despite triggering correction for every query, the average confidence gain was negligible (0.00), suggesting that the corrective retrieval mechanism did not substantially alter answer certainty.

Interestingly, CRAG demonstrated slightly lower average latency (7.05s) compared to RAG (7.92s), indicating potential inefficiencies in the baseline pipeline or measurement variability.

Overall, while CRAG enforced stricter quality control under the fixed threshold, it did not provide measurable confidence improvement in this implementation. Therefore, standard RAG offers comparable performance with lower architectural complexity, making it the more efficient choice under the current configuration.

2. Methodology

This section outlines the technical framework used to evaluate the RAG and CRAG architectures within the Prime Land Real Estate domain. We begin by describing the web crawling strategy used to collect structured and unstructured content from the Prime Lands website. Next, we explain the implementation of five different chunking strategies designed to optimize retrieval precision and contextual coherence. The Retrieval-Augmented Generation (RAG) pipeline architecture is then detailed, followed by the Cache Augmented Generation (CAG) caching strategy used to reduce redundant computation and improve efficiency. Finally, we present the Corrective RAG(CRAG) mechanism, including confidence-based correction logic and performance evaluation metrics used for comparison.

2.1 Web Crawling

The Prime Lands web crawler was implemented as an asynchronous data ingestion layer using Playwright (headless Chromium), Python async/await, and Beautiful Soup(bs4) to handle JavaScript-rendered content. Crawling was restricted to the domain https://www.primelands.lk with a maximum depth of 3 and a cap of 500 pages to prevent uncontrolled traversal. A Breadth-First Search (BFS) strategy was used to systematically explore property listings while maintaining depth control. Rate limiting (2 seconds between requests) and a 30-second timeout ensured polite and stable crawling. The should_crawl () method strictly filtered URLs to include only relevant property categories (/land, /houses, /apartment) while excluding media and non-content resources such as images, PDFs, CSS, JavaScript files, and other statis assets.

Content extraction was performed after dynamic rendering using Playwright's networkdle state.HTML was parsed with BeutifulSoup, and non-informative elements (scripts, styles, navigation bars, footers, iframes) were removed to isolate meaningful content. Structured metadata was extracted for property pages, including property ID (derived from URL), title, address, price, bedrooms, bathrooms, and amenities using pattern matching and semantic class detection. Cleaned textual content, headings, links, depth level, and metadata were stored in JSONL format to support downstream RAG indexing and analysis.

2.2 Chunking Strategies

Following the web crawling and structured content extraction phase, the next critical step in the pipeline is document chunking. Since the Prime Land Real Estate corpus contains long property descriptions, project details, and policy-related content, we implemented five different chunking strategies: **Fixed Chunking**, **Sliding Window Chunking, Parent-Child Chunking**, **Semantic Chunking**, **and Late Chunking**. The purpose of applying multiple strategies was to evaluate how different segmentation techniques affect retrieval precision, contextual coherence, and downstream RAG/CRAG performance. By transforming raw crawled documents into optimized retrieval units, this stage directly influences embedding quality, search relevance, and overall answer accuracy within the system.

Fixed Chunking

The fixed chunking strategy segments each crawled Prime Land document into uniform blocks of 800 tokens with an overlap of 100 tokens. The 800 token size was selected to balance contextual richness and embedding efficiency, ensuring that each chunk contains sufficient property details (price, location, amenities) without exceeding model input limits. A 100 token overlap was introduced to prevent information loss at chunk boundaries, particularly where key details such as pricing or specifications may span across sections. meaning, thereby supporting more stable and consistent performance in the downstream RAG pipeline.

Semantic Chunking

Semantic chunking divides documents into meaning-based segments with a target chunk size of 1000 tokens and a minimum chunk threshold of 200 tokens. Unlike fixed chunking, this method prioritizes logical content boundaries such as headings, feature descriptions, and property highlights extracted during crawling. The larger 1000 token upper limit ensures that semantically cohesive property descriptions remain intact, improving contextual embedding quality. The 200 token minimum prevents excessively small fragments that may weaken retrieval relevance. This configuration was chosen to preserve domain specific coherence in real estate listings, ensuring that complete contextual units such as full project description are indexed as meaningful retrieval candidates.

Sliding Window Chunking

The sliding window strategy uses a chunk size of 512 tokens with a 256 token overlap, meaning each new chunk shared 50% of its content with the previous one. The smaller 512 token size enhances retrieval precision by narrowing contextual scope, While the substantial overlap ensures continuity of information across boundaries. A 256 token overlap was specifically chosen to capture transitional details such as pricing tables or features lists that may span multiple paragraphs. This technique increases redundancy intentionally to improve recall in vector search, making it particularly useful for answering granular property specific questions within the Prime Land Real Estate corpus.

Parent-Child Chunking

Parent child chunks employ a hierarchical segmentation structure, where large parent chunks of 1200 tokens provide high-level contextual grouping, and smaller child chunks of 250 tokens (with 50 tokens overlap) are used for fine grained retrieval. The 1200 token parent size ensures comprehensive coverage of complete property listing or project description, while 250 token child chunks enhance retrieval specificity. The 50-token overlap maintains continuity between adjacent child segments, reducing boundary information loss. During retrieval, child chunks improve precision, while parent chunks restore broader context for generation. This configuration was selected to balance contextual depth and retrieval accuracy within the RAG and CRAG pipelines.

Late Chunking

Late chunking begins with a larger base segmentation of 1000 tokens, followed by dynamic splitting into smaller 300 token segments with a 150 token context window during retrieval. The 1000 token base size preserves semantic completeness during indexing, while the 300 token split size improves query-level precision. The 150 token context window ensures surrounding information is reintroduced when generating responses, preventing fragmentation of critical property details. This strategy was designed to combine the benefits of large-context indexing with fine-grained retrieval at inference time, thereby enhancing flexibility and improving contextual alignment in the Prime Land Real Estate knowledge base.

2.3 Retrieval Augmented Generation (RAG) Architecture

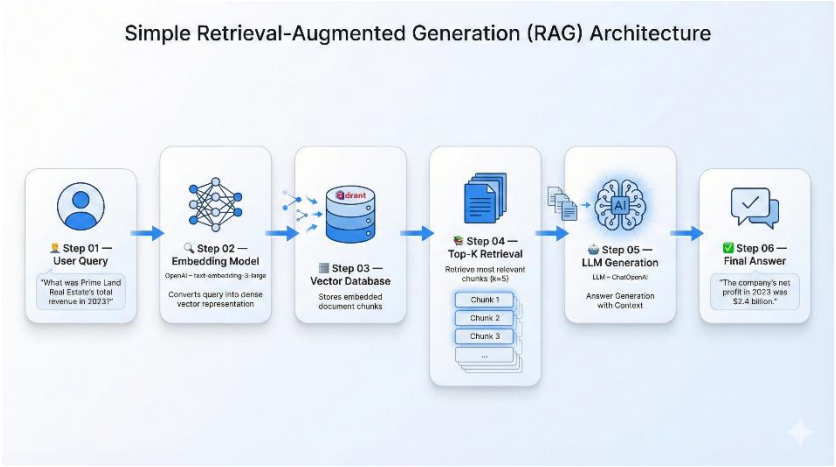

Figure 2.1 RAG Architecture for Prime Land Real Estate System

The RAG architecture was built using the **OpenAI text-embedding-large model**, which generates 3072-dimensional dense vectors to capture rich semantic representations of Prime Land Real Estate content. These embeddings were stored in a cloud-hosted **Qdrant** vector database to enable scalable and efficient similarity search. During retrieval, the system selects the Top 5 most similar chunk using a cosine similarity threshold of 0.5, ensuring only contextually relevant property information is passed to the language model.

```
PRIME_LAND_RAG_TEMPLATE="""You are an AI Information Assistant for Prime Lands Real Estate in Sri Lanka.

YOUR ROLE:
    - Provide accurate information about Prime Lands properties,locations,pricing,facilities(bedrooms and bathroomsm)
    - Help users understand project based ONLY on official Prime Lands content
    - Guide users to contact sales representatives when needed

GROUNDING RULES (CRITICAL):
    - Use ONLY the information provided in the CONTEXT section below
    - Cite sources inline using [URL] exactly as shown in the context
    - If specific information (price,availability,size,approval status,etc) is not available in the context,clearly state:
      "This information is not available in the provided data."
    - Do NOT assume,estimate,or fabricate property details
    - Do NOT provide legal,financial,or investment advice
    - Do NOT compare with competitors unless explicitly stated in the context

RESPONSE FORMAT (MANDATORY):

1. **Key Details**
 - 2-5 concise bullet points extracted directly from the context
 - Each point must reflect factual information
 - Include inline [URL] citations where applicable

2. **Answer**
 - Provide a clear and professional explanation addressing the user's question
 - Use inline [URL] citations for all factual claims
 - Keep it concise and structured

3. **Next Steps**
 - Suggest contacting Prime Lands for confirmation of pricing,availability,or site visits
 - Provide official contact  guidance (e.g., hotline,website inquiry form)

CONTEXT:
{context}

QUESTION:
{question}

Generate your response strictly following the required format above.

"""
```

Figure 2.2 RAG Prompt Template

The custom **PRIME_LAND_RAG_TEMPLATE** enforces strict grounding rules, requiring responses to rely solely on retrieved context, including inline URL citations, and avoid fabrication. This structured prompt design ensures factual accuracy, transparency, and domain-specific reliability in generated answers.

2.4 Cache Augmented Generation (CAG) Architecture

To improve efficiency and reduce operational cost, a **Cache-Augmented Generation (CAG)** layer was integrated into the RAG architecture. The cache stores previously generated responses and frequently asked questions to avoid repeated LLM calls for similar queries. A similarity threshold of 0.90 is used to detect paraphrased queries through embedding similarity, ensuring that only highly relevant matches trigger cache hits. The cache maintains a maximum capacity of 1000 entries, while dynamic conversation history expires after 24 hours to keep response fresh. Additionally, predefined FAQs stored in a FAQ.yaml file are preloaded into the cache, enabling instant responses to common Prime Lands queries and significantly reducing both latency and inference cost.

2.5 Corrective Retrieval Augmented Generation (CRAG)

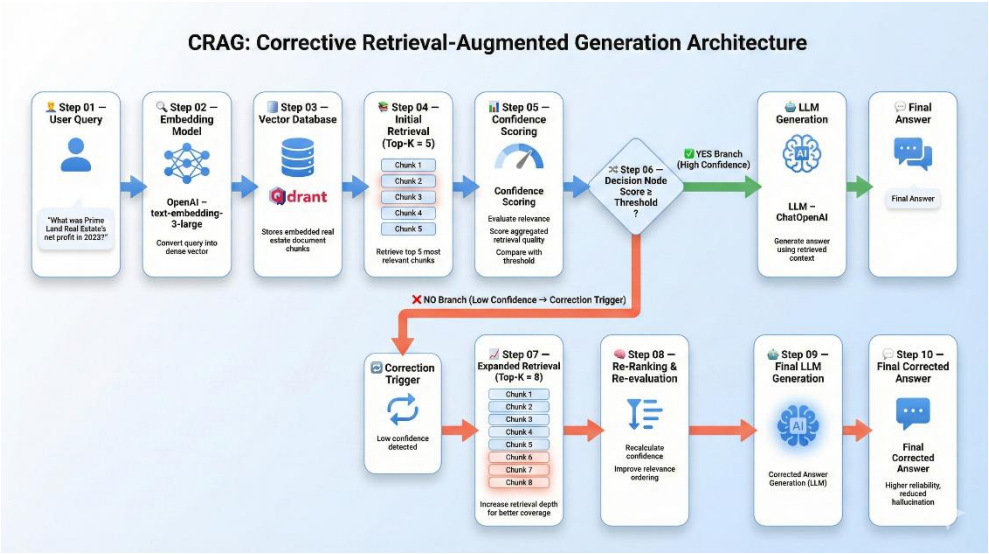

Figure 2.3 CRAG Architecture for Prime Land Real Estate System

The Corrective RAG(CRAG) architecture extends the baseline retrieval process by introducing a confidence driven validation layer. Initially, the system retrieves Top 5 chunks and computes a confidence score using a multi-factor heuristic. This score combines keyword overlap between query and documents (50% weight), content richness based on average document length normalized to 500 characters (30% weight), and chunking strategy diversity (20% weight). If the calculated confidence meets or exceeds the defined threshold of 0.6, generation proceeds directly to the LLM. Otherwise, retrieval is expanded to Top-8 chunks (expanded_k=8),and confidence is

calculated. If the first stage fails, a correction trigger counter increments to track corrective intervention frequency for evaluation analysis.